অধ্যায় ৪

08

Loops এবং Iteration

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- একই কাজ বারবার করার জন্য `for`, `while` এবং `do...while` Loop লিখতে।
- কোন অবস্থায় কোন Loop উপযুক্ত, তা বেছে নিতে।
- একটি Array-র প্রতিটি মান Index ধরে পড়তে।
- Loop-এর শুরু ও শেষের সীমা ঠিক রেখে Off-by-One Error এড়াতে।
- `break` ও `continue` দিয়ে Loop-এর গতিপথ নিয়ন্ত্রণ করতে।
- Nested Loop লিখতে এবং তার খরচ বুঝতে।
- Infinite Loop চিনতে, থামাতে এবং প্রতিরোধ করতে।
- Loop দিয়ে যোগফল, গণনা, সর্বোচ্চ ও সর্বনিম্ন বের করতে।
- একটি সম্পূর্ণ **Cart Analyzer** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৩ সম্পন্ন করা: Comparison ও Logical Operator, `if` / `else`, `+=` এবং `++`।
- অধ্যায় ২-এর Object-এর প্রাথমিক পরিচয়: `{ name: 'Pen' }` এবং ডট দিয়ে তথ্য পড়া।
- `chapter-04` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ৩-এর Discount Calculator একটি পণ্যের হিসাব করত। বাস্তব Cart-এ পাঁচটি, পঞ্চাশটি বা পাঁচশটি পণ্য থাকতে পারে। প্রতিটির জন্য আলাদা লাইন লেখা সম্ভব নয়, আর আগে থেকে জানাও যায় না কয়টি পণ্য থাকবে।

Loop এই সমস্যার সমাধান: একটি কাজ একবার লিখে যতবার প্রয়োজন ততবার চালানো। তালিকার প্রতিটি পণ্যের দাম যোগ করা, প্রতিটি শিক্ষার্থীর ফল দেখানো, একটি শর্ত পূরণ না হওয়া পর্যন্ত চেষ্টা করে যাওয়া—সবই Loop-এর কাজ। Program যে গতিতে হাজার হাজার তথ্য সামলাতে পারে, তার ভিত্তি এটিই।

Loop-এর সঙ্গে দুটি নতুন ধরনের ভুলও আসে: একবার বেশি বা একবার কম চলা, এবং কখনো না থামা। এই অধ্যায়ে দুটোই চিনে নেবেন।

৪.১ Loop কেন প্রয়োজন

পাঁচটি পণ্যের নম্বর দেখাতে হলে পাঁচবার `console.log()` লেখা যায়। পাঁচশটি হলে? Loop একই কাজ এভাবে করে:

first-loop.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (let i = 1; i <= 5; i++) {
  console.log(`পণ্য ${i}`);
}
```

Output

```
পণ্য 1
পণ্য 2
পণ্য 3
পণ্য 4
পণ্য 5
```

`5`-এর জায়গায় `500` লিখলেই পাঁচশ লাইন দেখাবে; Code এক লাইনও বাড়বে না। Loop-এর ভেতরের Code একবার চলাকে একটি **Iteration** বলে। ওপরের Loop-এ পাঁচটি Iteration হয়েছে।

৪.২ for Loop

কতবার চলবে তা আগে থেকে জানা থাকলে `for` সবচেয়ে উপযুক্ত। এর প্রথম বন্ধনীতে Semicolon দিয়ে আলাদা করা তিনটি অংশ থাকে।

অংশ	উদাহরণ	কখন চলে	কাজ
Initialization	`let i = 1`	শুরুতে একবার	গণনার Variable তৈরি
Condition	`i <= 5`	প্রতিটি Iteration-এর আগে	সত্য হলে Loop চলে, মিথ্যা হলে থামে
Update	`i++`	প্রতিটি Iteration-এর পরে	Variable বদলায়

গণনার Variable-টিকে **Counter** বলা হয়। `i` নামটি একটি পুরোনো রীতি; ছোট Loop-এ এটি গ্রহণযোগ্য।

ধাপগুলো একটি একটি করে অনুসরণ করলে Loop-এর আচরণ পরিষ্কার হয়।

sum-to-ten.js · JavaScript — চলবে: Browser Console ও Node.js

```
let sum = 0;

for (let i = 1; i <= 10; i++) {
  sum += i;
}

console.log(sum);
```

Output

```
55
```

Iteration	`i`	শর্ত `i <= 10`	`sum` (Iteration-এর পরে)
১	1	সত্য	1
২	2	সত্য	3
৩	3	সত্য	6
…	…	…	…
১০	10	সত্য	55
—	11	মিথ্যা	Loop শেষ

এ ধরনের টেবিলকে Trace Table বলে। Loop প্রত্যাশামতো কাজ না করলে হাতে এমন একটি টেবিল তৈরি করা সবচেয়ে নির্ভরযোগ্য Debugging কৌশলগুলোর একটি।

`sum` Loop-এর বাইরে তৈরি করা হয়েছে, কারণ এটি সব Iteration জুড়ে মান ধরে রাখে। Loop-এর ভেতরে তৈরি করলে প্রতিবার নতুন করে শূন্য হয়ে যেত। আর `i` শুধু Loop-এর ভেতরেই কার্যকর; Loop শেষ হলে সেটি আর থাকে না।

ভিন্ন ধাপে এবং উল্টো দিকে

Update অংশে যেকোনো পরিবর্তন লেখা যায়।

steps.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (let i = 2; i <= 10; i += 2) {
  console.log(i);
}

for (let i = 3; i >= 1; i--) {
  console.log(`বাকি ${i}`);
}
```

Output

```
2
4
6
8
10
বাকি 3
বাকি 2
বাকি 1
```

উল্টো দিকে গোনার সময় শর্তটিও উল্টে যায়: `i >= 1`, এবং Update হয় `i--`।

৪.৩ Array: তালিকার প্রথম পরিচয়

Loop-এর সবচেয়ে বড় ব্যবহার তালিকা নিয়ে কাজ করা। JavaScript-এ একাধিক মানকে ক্রমানুসারে একসঙ্গে রাখার মান হলো **Array**। তৃতীয় বন্ধনীর ভেতরে কমা দিয়ে আলাদা করে মানগুলো লেখা হয়। Array নিয়ে পূর্ণ আলোচনা অধ্যায় ৬-এ; এখানে শুধু Loop-এর জন্য যতটুকু দরকার।

array-intro.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];

console.log(prices[0]);
console.log(prices[2]);
console.log(prices.length);
console.log(prices[3]);
```

Output

```
120
950
3
undefined
```

- Array-র প্রতিটি মানকে একটি **Element** বলে, আর তার অবস্থানের নম্বরকে **Index**।
- **Index শূন্য থেকে শুরু হয়।** প্রথম Element `prices[0]`, দ্বিতীয়টি `prices[1]`।
- `prices.length` Element-এর সংখ্যা দেয়।
- শেষ Element-এর Index তাই `length - 1`। এখানে তিনটি Element, শেষ Index `2`।
- যে Index-এ কিছু নেই, সেখান থেকে পড়লে Error হয় না; `undefined` পাওয়া যায়।

শেষ দুটি বিষয় মনে রাখুন; অনুচ্ছেদ ৪.৪-এর ভুলটি এখান থেকেই আসে।

Loop দিয়ে Array পড়া

loop-array.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];
let total = 0;

for (let i = 0; i < prices.length; i++) {
  console.log(`Index ${i}: ${prices[i]}`);
  total += prices[i];
}

console.log(`মোট: ${total}`);
```

Output

```
Index 0: 120
Index 1: 15
Index 2: 950
মোট: 1085
```

এই ধরনটি মুখস্থ রাখার মতো: `let i = 0` থেকে শুরু, `i < array.length` পর্যন্ত, `i++` করে এগোনো। Array-তে আরও Element যোগ হলেও Loop বদলাতে হবে না, কারণ শর্তটি `length` থেকে নিজেই সীমা জেনে নেয়।

8.8 Loop-এর সীমা এবং Off-by-One Error

Loop একবার বেশি বা একবার কম চললে যে ভুল হয়, তাকে **Off-by-One Error** বলে। এটি Programming-এর সবচেয়ে প্রচলিত ভুলগুলোর একটি, এবং প্রায় সব সময় দুটি জায়গা থেকে আসে: শুরুর মান এবং শর্তের Operator।

off-by-one.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];
let total = 0;

for (let i = 0; i <= prices.length; i++) {
  console.log(prices[i]);
  total += prices[i];
}

console.log(total);
```

Output

```
120
15
950
undefined
NaN
```

শর্তে `<`-এর জায়গায় `<=` লেখায় Loop চারবার চলেছে। চতুর্থবার `i` হয়েছে `3`, আর `prices[3]`-এ কিছু নেই। `undefined` যোগ করায় পুরো যোগফল `NaN` হয়ে গেছে। কোনো Error Message আসেনি; শুধু ফল ভুল।

উল্টো ভুলটিও ঘটে: `let i = 1` দিয়ে শুরু করলে প্রথম Element বাদ পড়ে।

লেখা	কোন Index-গুলো পড়া হয় (৩টি Element)	ফল
`i = 0; i < length`	0, 1, 2	সঠিক
`i = 0; i <= length`	0, 1, 2, 3	একবার বেশি; শেষে `undefined`
`i = 1; i < length`	1, 2	প্রথমটি বাদ
`i = 0; i < length - 1`	0, 1	শেষটি বাদ

> **মনে রাখুন:** Index শূন্য থেকে শুরু, তাই Array-র Loop-এ `i = 0` এবং `i < array.length`। মানুষের জন্য ক্রমিক নম্বর দেখাতে হলে দেখানোর সময় `i + 1` লিখুন; Loop-এর সীমা বদলাবেন না।

সীমা ঠিক আছে কি না যাচাইয়ের সহজ উপায়: নিজেকে তিনটি প্রশ্ন করুন। প্রথম Iteration-এ Counter-এর মান কত? শেষ Iteration-এ কত? মোট কয়বার চলবে?

৪.৫ while Loop

কতবার চলতে হবে তা আগে থেকে জানা না থাকলে `while` ব্যবহার করা হয়। এতে শুধু একটি শর্ত থাকে; শর্ত যতক্ষণ সত্য, Loop চলে।

savings.js · JavaScript চলবে: Browser Console ও Node.js

```
const target = 5000;
const monthlySaving = 1200;
let balance = 0;
let months = 0;

while (balance < target) {
  balance += monthlySaving;
  months++;
}

console.log(`${months} মাসে জমা হবে ${balance} টাকা`);
```

Output

```
5 মাসে জমা হবে 6000 টাকা
```

প্রশ্নটি ছিল "কত মাস লাগবে?"—উত্তরটিই অজানা, তাই `for` দিয়ে আগে থেকে বলা যায় না কতবার চলবে। `while` প্রতিবার শর্ত দেখে: লক্ষ্যে পৌঁছানো হয়নি? তাহলে আরেক মাস।

`for`-এর তিনটি অংশ `while`-এও থাকে, শুধু আলাদা জায়গায়: Variable তৈরি Loop-এর আগে, শর্ত বন্ধনীতে, আর Update Loop-এর ভেতরে। Update-টি লিখতে ভুলে গেলে শর্ত কখনো মিথ্যা হয় না; বিষয়টি অনুচ্ছেদ ৪.৯-এ আছে।

	for	while
উপযুক্ত	Iteration-এর সংখ্যা জানা; Array ধরে চলা	সংখ্যা অজানা; একটি অবস্থা না হওয়া পর্যন্ত চলা
Counter	Loop-এর শিরোনামেই	নিজে আলাদাভাবে সামলাতে হয়

৪.৬ do...while Loop

`while` আগে শর্ত দেখে, তারপর চলে; শর্ত শুরুতেই মিথ্যা হলে একবারও চলে না। `do...while` উল্টো: আগে একবার চলে, তারপর শর্ত দেখে।

do-while.js · JavaScript চলবে: Browser Console ও Node.js

```
const stock = 0;

while (stock > 0) {
  console.log('while: একবার চলল');
}

do {
  console.log('do...while: একবার চলল');
} while (stock > 0);
```

Output

```
do...while: একবার চলল
```

শর্ত দুই ক্ষেত্রেই মিথ্যা, কিন্তু `do...while`-এর ভেতরের Code একবার চলেছে। `while`-এর বন্ধনীর পরে Semicolon লক্ষ করুন; `do...while`-এ এটি প্রয়োজন।

যে কাজ অন্তত একবার করতেই হবে, সেখানে এটি উপযুক্ত: ব্যবহারকারীকে একটি প্রশ্ন করা এবং উত্তর সঠিক না হওয়া পর্যন্ত আবার করা; একটি Menu দেখানো; একটি সংযোগের চেষ্টা করা।

attempts.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const maxAttempts = 3;
let attempt = 0;

do {
  attempt++;
  console.log(`চেষ্টা ${attempt}`);
} while (attempt < maxAttempts);
```

Output

```
চেষ্টা 1
চেষ্টা 2
চেষ্টা 3
```

বাস্তবে `do...while` তুলনামূলক কম লাগে। অধিকাংশ ক্ষেত্রে `for` বা `while`-ই যথেষ্ট।

৪.৭ break এবং continue

Loop-এর স্বাভাবিক গতি বদলানোর দুটি উপায় আছে।

- **`break`** — Loop সম্পূর্ণ থামিয়ে দেয়। Loop-এর পরের লাইন থেকে Program চলতে থাকে।
- **`continue`** — শুধু বর্তমান Iteration-এর বাকি অংশ বাদ দেয়, এবং পরের Iteration-এ চলে যায়।

break: পাওয়া গেলে থেমে যাওয়া

break.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950, 280, 1500];
let checked = 0;

for (let i = 0; i < prices.length; i++) {
  checked++;

  if (prices[i] > 500) {
    console.log(`প্রথম দামি পণ্য: ${prices[i]} (Index ${i})`);
    break;
  }
}

console.log(`যাচাই করা হয়েছে ${checked}টি`);
```

Output

```
প্রথম দামি পণ্য: 950 (Index 2)
যাচাই করা হয়েছে 3টি
```

যা খোঁজা হচ্ছিল তা তৃতীয় Element-এই পাওয়া গেছে, তাই বাকি দুটি আর দেখা হয়নি। তালিকা বড় হলে এই সাশ্রয় তাৎপর্যপূর্ণ।

continue: অপ্রয়োজনীয়টি বাদ দেওয়া

continue.js · JavaScript — চলবে: Browser Console ও Node.js

```
const amounts = [50, -10, 30, 0, 20];
let total = 0;
let skipped = 0;

for (let i = 0; i < amounts.length; i++) {
  if (amounts[i] <= 0) {
    skipped++;
    continue;
  }

  total += amounts[i];
}

console.log(`মোট: ${total}, বাদ পড়েছে: ${skipped}`);
```

Output

```
মোট: 100, বাদ পড়েছে: 2
```

শূন্য বা ঋণাত্মক মান পেলে `continue` যোগ করার লাইনটি এড়িয়ে পরের Element-এ চলে যায়। এটি অধ্যায় ৩-এর Guard Clause-এরই Loop-সংস্করণ: অগ্রহণযোগ্য অবস্থা আগে সরিয়ে দিলে মূল কাজ `if` -এর ভেতরে ঢুকে যায় না।

> **সতর্কতা:** `while` Loop-এ `continue` ব্যবহারের সময় খেয়াল রাখুন, Counter-এর Update যেন `continue` -এর আগে থাকে। নইলে Update বাদ পড়ে এবং Loop আর থামে না। `for` Loop-এ এই ঝুঁকি নেই, কারণ `continue` -এর পরেও Update অংশটি চলে।

৪.৮ Nested Loop

একটি Loop-এর ভেতরে আরেকটি Loop থাকলে তাকে **Nested Loop** বলে। বাইরের Loop-এর প্রতিটি Iteration-এ ভেতরের Loop শুরু থেকে শেষ পর্যন্ত পুরোটা চলে।

nested.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (let row = 1; row <= 3; row++) {
  let line = '';

  for (let column = 1; column <= 4; column++) {
    line += `${row * column} `;
  }

  console.log(line);
}
```

Output

```
1 2 3 4
2 4 6 8
3 6 9 12
```

বাইরের Loop তিনবার চলে, আর প্রতিবার ভেতরের Loop চারবার। ভেতরের লাইনটি মোট ৩ × ৪ = ১২ বার চলেছে। `line` বাইরের Loop-এর ভেতরে কিন্তু ভেতরের Loop-এর বাইরে তৈরি, তাই প্রতিটি সারির জন্য এটি নতুন করে ফাঁকা হয়।

দুটি Loop-এ আলাদা Counter ব্যবহার করুন এবং অর্থবহ নাম দিন: `row` ও `column` পড়লেই বোঝা যায় কোনটি কী। দুটোতেই `i` লিখলে ভেতরেরটি বাইরেরটিকে ঢেকে দেয়।

Nested Loop-এর খরচ

Nested Loop-এ কাজের পরিমাণ গুণ হয়ে বাড়ে।

তালিকার আকার	একটি Loop	দুটি Nested Loop
10	10 বার	100 বার

তালিকার আকার	একটি Loop	দুটি Nested Loop
1,000	1,000 বার	10,00,000 বার
1,00,000	1,00,000 বার	1,000 কোটি বার

ছোট তালিকায় পার্থক্য বোঝা যায় না; বড় তালিকায় Program কয়েক সেকেন্ড থেকে কয়েক ঘণ্টায় চলে যেতে পারে। Nested Loop লেখার আগে ভাবুন: ভেতরের Loop-টি কি সত্যিই প্রয়োজন, নাকি একটি Loop-এই কাজ হয়? Performance নিয়ে বিস্তারিত আলোচনা অধ্যায় ১৯-এ আছে।

৪.৯ Infinite Loop এবং তার প্রতিরোধ

যে Loop-এর শর্ত কখনো মিথ্যা হয় না, সেটি থামে না। একে **Infinite Loop** বলে। Program আটকে যায়, Computer-এর Processor পুরো ব্যস্ত থাকে, এবং Browser-এ হলে Page সাড়া দেওয়া বন্ধ করে।

নিচের Code-টি একটি Infinite Loop। **এটি চালানোর দরকার নেই;** চালিয়ে ফেললে Terminal-এ `Ctrl + C` চেপে থামান।

infinite.js · JavaScript — চলবে: Node.js

```
let i = 0;

while (i !== 10) {
  i += 3;
}

console.log('এই লাইনে কখনো পৌঁছানো হবে না');
```

`i`-এর মান হয় 3, 6, 9, 12, 15...। এটি কখনো ঠিক `10` হয় না, তাই `i !== 10` সব সময় সত্য। শর্তটি `i < 10` হলে `12`-তে গিয়ে Loop থেমে যেত।

Infinite Loop-এর তিনটি সাধারণ কারণ:

কারণ	উদাহরণ	সমাধান
Update নেই	`while (i < 5) { console.log(i); }`	Loop-এর ভেতরে `i++`
Update ভুল দিকে	`for (let i = 0; i < 5; i--)`	`i++`
শর্ত ঠিক একটি মানের অপেক্ষায়	`while (i !== 10)` যখন `i` তিন করে বাড়ে	`!==`-এর বদলে `<` বা `>`

নিরাপত্তা-সীমা

যে `while` Loop-এ শর্ত বাইরের কোনো অবস্থার ওপর নির্ভর করে, সেখানে সর্বোচ্চ Iteration-এর একটি সীমা বসানো একটি ভালো অভ্যাস।

safety-limit.js · JavaScript — চলবে: Browser Console ও Node.js

```
const MAX_STEPS = 1000;
let i = 0;
let steps = 0;

while (i !== 10) {
  i += 3;
  steps++;

  if (steps >= MAX_STEPS) {
    console.log('থামানো হলো: শর্তটি মিথ্যা হচ্ছে না');
    break;
  }
}

console.log(`মোট ধাপ: ${steps}`);
```

Output

```
থামানো হলো: শর্তটি মিথ্যা হচ্ছে না
মোট ধাপ: 1000
```

একই ভুল শর্ত, কিন্তু এবার Program আটকে না থেকে সমস্যাটি জানিয়ে দিয়েছে। সীমাটি ভুলকে লুকায় না; বরং ধরা সহজ করে।

৪.১০ Loop দিয়ে Data প্রক্রিয়াকরণ

Loop-এর অধিকাংশ কাজ কয়েকটি পরিচিত ধরনের মধ্যে পড়ে। ধরনগুলো চিনে রাখলে নতুন সমস্যায় কোনটি লাগবে তা সহজে বোঝা যায়। প্রতিটিতে Loop-এর আগে একটি Variable তৈরি হয়, যাতে ফল জমা হয়; একে **Accumulator** বলে।

patterns.js · JavaScript — চলবে: Browser Console ও Node.js

```
const marks = [72, 88, 45, 91, 60];

let sum = 0;
let passed = 0;
let highest = marks[0];
let lowest = marks[0];

for (let i = 0; i < marks.length; i++) {
  const mark = marks[i];
```

```
  sum += mark;

  if (mark >= 60) {
    passed++;
  }

  if (mark > highest) {
    highest = mark;
  }

  if (mark < lowest) {
    lowest = mark;
  }
}

console.log(`যোগফল: ${sum}`);
console.log(`গড়: ${sum / marks.length}`);
console.log(`৬০ বা বেশি: ${passed} জন`);
console.log(`সর্বোচ্চ: ${highest}, সর্বনিম্ন: ${lowest}`);
```

Output

```
যোগফল: 356
গড়: 71.2
৬০ বা বেশি: 4 জন
সর্বোচ্চ: 91, সর্বনিম্ন: 45
```

ধরন	শুরুর মান	প্রতিটি Iteration-এ
যোগফল	`0`	`sum += value`
গণনা	`0`	শর্ত মিললে `count++`
সর্বোচ্চ	প্রথম Element	বড় হলে বদলে দেওয়া
সর্বনিম্ন	প্রথম Element	ছোট হলে বদলে দেওয়া
খোঁজা	`null` বা `-1`	পাওয়া গেলে রেখে `break`

সর্বোচ্চ ও সর্বনিম্নের শুরুর মান হিসেবে `0` নয়, প্রথম Element নেওয়া হয়েছে। `0` দিয়ে শুরু করলে সব নম্বর ঋণাত্মক হলে সর্বোচ্চ ভুল আসত, আর সর্বনিম্ন সব সময় `0`-ই থেকে যেত।

একটি Loop-এই চারটি হিসাব করা হয়েছে। চারটি আলাদা Loop লিখলেও ফল একই হতো, কিন্তু তালিকাটি চারবার পড়তে হতো।

Object-এর তালিকা

বাস্তব তথ্য সাধারণত Object-এর Array হিসেবে আসে: প্রতিটি Element একটি Object।

students.js · JavaScript — চলবে: Browser Console ও Node.js

```
const students = [
  { name: 'Sara', mark: 80 },
  { name: 'Rafi', mark: 28 },
  { name: 'Mina', mark: 65 },
];

for (let i = 0; i < students.length; i++) {
  const student = students[i];
  const status = student.mark >= 33 ? 'উত্তীর্ণ' : 'অনুত্তীর্ণ';

  console.log(`${i + 1}. ${student.name}: ${status}`);
}
```

Output

```
1. Sara: উত্তীর্ণ
2. Rafi: অনুত্তীর্ণ
3. Mina: উত্তীর্ণ
```

`students[i]` একটি Object দেয়; সেটিকে `student` নামে রেখে ডট দিয়ে তার তথ্য পড়া হয়েছে। ক্রমিক নম্বর দেখাতে `i + 1` ব্যবহার করা হয়েছে, কিন্তু Loop-এর সীমা শূন্য থেকেই। Project-এ ঠিক এই ধরনটিই ব্যবহার করা হবে।

৪.১১ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
শর্তে `<=` (`i <= array.length`)	শেষে `undefined`; যোগফল `NaN`	`i < array.length`
Index `1` থেকে শুরু	প্রথম Element বাদ পড়ে	`let i = 0`
`while`-এ Update নেই	Program আটকে যায়	Loop-এর ভেতরে Counter বদলান
Accumulator Loop-এর ভেতরে তৈরি	প্রতিবার শূন্য হয়ে যায়; শুধু শেষ মানটি থাকে	Loop-এর আগে তৈরি করুন
Counter `const` দিয়ে তৈরি	`TypeError: Assignment to constant variable.`	`let i`
Nested Loop-এ একই Counter	বাইরের Loop এলোমেলো চলে	আলাদা নাম: `row`, `column`
সর্বোচ্চ খুঁজতে `0` দিয়ে শুরু	ঋণাত্মক সংখ্যায় ভুল ফল	প্রথম Element দিয়ে শুরু
`break`-এর বদলে `continue`, বা উল্টো	খোঁজা থামে না, অথবা আগেই থেমে যায়	`break` থামায়, `continue` এড়িয়ে যায়

ভুল	কী দেখা যায়	সমাধান
Loop-এর ভেতরে Array-র বাইরের Index	`undefined`	Index সব সময় `0` থেকে `length - 1`

৪.১২ Debugging নির্দেশিকা

Loop-এর Bug ধরার মূল উপায় হলো প্রতিটি Iteration-এ কী ঘটছে তা দেখা।

debug-loop.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];
let total = 0;

for (let i = 0; i < prices.length; i++) {
  total += prices[i];
  console.log(`i=${i} মান=${prices[i]} total=${total}`);
}
```

Output

```
i=0 মান=120 total=120
i=1 মান=15 total=135
i=2 মান=950 total=1085
```

এটি Trace Table-এর স্বয়ংক্রিয় রূপ। Counter, বর্তমান মান এবং Accumulator—তিনটি একসঙ্গে দেখলে বোঝা যায় কোন Iteration-এ হিসাব ভুল পথে গেছে।

Loop প্রত্যাশামতো কাজ না করলে:

১. Counter, বর্তমান Element এবং Accumulator প্রতি Iteration-এ দেখান।
২. প্রথম এবং শেষ Iteration আলাদাভাবে লক্ষ করুন; Off-by-One সেখানেই ধরা পড়ে।
৩. Iteration-এর সংখ্যা গুনুন: এটি কি `array.length`-এর সমান?
৪. ফল `NaN` হলে খুঁজুন কোন Iteration-এ `undefined` বা String যোগ হয়েছে।
৫. Program আটকে গেলে `Ctrl + C` চেপে থামান, তারপর দেখুন Counter আদৌ বদলাচ্ছে কি না এবং সঠিক দিকে বদলাচ্ছে কি না।
৬. ছোট তালিকা দিয়ে পরীক্ষা করুন: শূন্য, এক এবং দুই Element-এর Array।
৭. ফাঁকা Array দিলে কী হয় দেখুন। `marks[0]` তখন `undefined`, এবং গড়ের হিসাবে শূন্য দিয়ে ভাগ হয়।

৪.১৩ পেশাদার অভ্যাস

- **সঠিক Loop বেছে নিন।** সংখ্যা জানা থাকলে বা Array ধরে চললে `for`; একটি অবস্থার অপেক্ষায় থাকলে `while`।
- **সীমা `length` থেকে নিন।** `i < 3` নয়, `i < prices.length`।
- **Counter-এর অর্থবহ নাম দিন** যখন Loop ছোট নয় বা Nested: `row`, `column`, `attempt`।
- **Loop-এর ভেতরে বর্তমান Element-কে একটি নাম দিন।** `const item = items[i];` লিখলে বাকি Code পড়া সহজ হয়।
- **Loop-এর ভেতরের কাজ ছোট রাখুন।** বড় হলে একটি Function-এ নিয়ে যান।
- **যে `while`-এর শেষ নিশ্চিত নয়, তাতে নিরাপত্তা-সীমা দিন।**
- **Loop চলাকালে যে Array ধরে চলছেন সেটি বদলাবেন না।** Element যোগ বা বাদ দিলে Index সরে যায়।
- **ফাঁকা তালিকার কথা ভাবুন।** শূন্য Element-এ আপনার Code কী করবে?
- **অপ্রয়োজনীয় Nested Loop এড়িয়ে চলুন।**

৪.১৪ Project: Cart Analyzer

একটি Cart-এর সব পণ্য বিশ্লেষণ করে সারসংক্ষেপ তৈরি করবেন: মোট টাকা, মোট পণ্যের সংখ্যা, সবচেয়ে বড় অঙ্কের পণ্য এবং একটি নির্দিষ্ট অঙ্কের বেশি দামের পণ্যগুলোর নাম।

Project-এর চাহিদা

- Cart হলো Object-এর একটি Array। প্রতিটি Object-এ পণ্যের নাম (`name`), একটির দাম (`price`) এবং পরিমাণ (`quantity`) আছে।
- একটি পণ্য গ্রহণযোগ্য, যদি নাম ফাঁকা না হয়, দাম শূন্য বা তার বেশি পূর্ণসংখ্যা হয়, এবং পরিমাণ ১ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা হয়।
- অগ্রহণযোগ্য পণ্য হিসাবে ধরা হবে না, তবে কয়টি বাদ পড়ল তা জানাতে হবে।
- প্রতিটি পণ্যের জন্য দাম × পরিমাণ দেখাতে হবে।
- মোট টাকা, মোট পরিমাণ, সবচেয়ে বড় অঙ্কের পণ্য এবং একটি নির্দিষ্ট সীমার সমান বা বেশি অঙ্কের পণ্যগুলোর নাম দেখাতে হবে।
- ফাঁকা Cart দিলে Program ভেঙে পড়বে না।

File-এর কাঠামো

Folder-এর কাঠামো

```
cart-analyzer/
├── cart.js
├── README.md
└── tests/
    └── cart.test.js
```

cart.js

File-টি তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` লাইনগুলো লিখতে হবে না।

ধাপ ১: একটি পণ্য যাচাই

cart.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
'use strict';

const MAX_QUANTITY = 100;

// একটি পণ্য গ্রহণযোগ্য কি না যাচাই করে।
function isValidItem(item) {
  const hasName = item.name.trim() !== '';
  const hasPrice = Number.isInteger(item.price) && item.price >= 0;
  const hasQuantity = Number.isInteger(item.quantity) &&
    item.quantity >= 1 && item.quantity <= MAX_QUANTITY;

  return hasName && hasPrice && hasQuantity;
}

// ...
```

তিনটি শর্তের প্রতিটিকে একটি নাম দেওয়া হয়েছে, তারপর `&&` দিয়ে জোড়া লাগানো হয়েছে। এক লাইনে সব লিখলে একই কাজ হতো, কিন্তু পড়া কঠিন হতো। একটি Expression দীর্ঘ হলে Operator-এর পরে ভেঙে পরের লাইনে লেখা যায়, যেমন `hasQuantity`-তে করা হয়েছে।

ধাপ ২: পুরো Cart বিশ্লেষণ

cart.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// Cart-এর সব পণ্য একবার ঘুরে একটি সারসংক্ষেপ তৈরি করে।
function analyzeCart(items, threshold) {
  let total = 0;
  let units = 0;
  let skipped = 0;
```

```
  let highestName = '';
  let highestAmount = 0;
  let bigItems = '';

  for (let i = 0; i < items.length; i++) {
    const item = items[i];

    if (!isValidItem(item)) {
      skipped++;
      continue;
    }

    const amount = item.price * item.quantity;

    total += amount;
    units += item.quantity;

    if (amount > highestAmount) {
      highestAmount = amount;
      highestName = item.name;
    }

    if (amount >= threshold) {
      bigItems += bigItems === '' ? item.name : `, ${item.name}`;
    }
  }

  return {
    total: total,
    units: units,
    skipped: skipped,
    highestName: highestName,
    highestAmount: highestAmount,
    bigItems: bigItems,
  };
}

// ...
```

এই Function-এ অনুচ্ছেদ ৪.১০-এর প্রায় সব ধরন একসঙ্গে আছে: যোগফল (`total`, `units`), গণনা (`skipped`), সর্বোচ্চ (`highestAmount`), এবং অগ্রহণযোগ্য পণ্য বাদ দিতে `continue`। ফলগুলো একটি Object-এ ভরে ফেরত দেওয়া হয়েছে, কারণ একটি Function একটিই মান ফেরত দিতে পারে; একাধিক তথ্য পাঠাতে সেগুলোকে একটি Object-এ রাখতে হয়।

ধাপ ৩: Report দেখানো

cart.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// প্রতিটি পণ্যের লাইন এবং শেষে সারসংক্ষেপ দেখায়।
function printReport(items, threshold) {
  for (let i = 0; i < items.length; i++) {
    const item = items[i];
    const label = `${i + 1}. ${item.name}`;
    const amount = item.price * item.quantity;

    console.log(`${label}: ${item.price} x ${item.quantity} = ${amount}`);
  }

  const summary = analyzeCart(items, threshold);

  console.log(`মোট: ${summary.total} টাকা, পরিমাণ: ${summary.units}`);
  console.log(`সবচেয়ে বড় অঙ্ক: ${summary.highestName} (${summary.highestAmount})`);
  console.log(`${threshold} টাকা বা বেশি: ${summary.bigItems}`);
  console.log(`বাদ পড়েছে: ${summary.skipped}টি`);
}

const cart = [
  { name: 'Notebook', price: 120, quantity: 3 },
  { name: 'Pen', price: 15, quantity: 10 },
  { name: 'School Bag', price: 950, quantity: 1 },
  { name: 'Calculator', price: 650, quantity: 0 },
  { name: 'Water Bottle', price: 280, quantity: 2 },
];

printReport(cart, 500);
```

Output

```
1. Notebook: 120 x 3 = 360
2. Pen: 15 x 10 = 150
3. School Bag: 950 x 1 = 950
4. Calculator: 650 x 0 = 0
5. Water Bottle: 280 x 2 = 560
মোট: 2020 টাকা, পরিমাণ: 16
সবচেয়ে বড় অঙ্ক: School Bag (950)
500 টাকা বা বেশি: School Bag, Water Bottle
বাদ পড়েছে: 1টি
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **সঠিক সীমা।** দুটি Loop-ই `let i = 0` থেকে `i < items.length` পর্যন্ত। ক্রমিক নম্বরের জন্য দেখানোর সময় `i + 1`।

- `const item = items[i];` — প্রতিটি Iteration-এ বর্তমান পণ্যটিকে একটি নাম দেওয়া হয়েছে। `const` এখানে ঠিক আছে, কারণ প্রতিটি Iteration-এ এটি নতুন করে তৈরি হয়।
- `continue` **দিয়ে Guard।** Calculator-এর পরিমাণ শূন্য, তাই `isValidItem` মিথ্যা দেয়; `skipped` এক বাড়ে এবং বাকি হিসাব বাদ পড়ে। মোট টাকা ও পরিমাণে এটি ধরা হয়নি।
- **সর্বোচ্চ খোঁজা।** এখানে শুরুর মান `0` নেওয়া নিরাপদ, কারণ যাচাইয়ের পর কোনো অঙ্ক ঋণাত্মক হতে পারে না। দুটি পণ্যের অঙ্ক সমান হলে প্রথমটিই থাকে, কারণ শর্তটি `>`, `>=` নয়।
- **নামের তালিকা তৈরি।** `bigItems` একটি String, যাতে নামগুলো জোড়া লাগে। প্রথম নামের আগে কমা বসবে না, তাই Ternary দিয়ে দেখা হয়েছে String-টি এখনো ফাঁকা কি না। অধ্যায় ৬-এ Array দিয়ে এ কাজ আরও সহজে করা যাবে।
- **হিসাব আর দেখানো আলাদা।** `analyzeCart` কিছু দেখায় না; শুধু হিসাব করে ফেরত দেয়। `printReport` দেখানোর কাজ করে। এই বিভাজনের কারণে `analyzeCart`-কে আলাদাভাবে পরীক্ষা করা যায়।

`printReport` প্রতিটি পণ্যের লাইন দেখায়, অগ্রহণযোগ্যটিসহ; বাদ পড়ার বিষয়টি শুধু সারসংক্ষেপে ধরা পড়ে। অনুশীলনী ১১-তে এটি উন্নত করবেন।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node cart.js
```

অবস্থা	যা দেখা যাওয়ার কথা
ওপরের পাঁচটি পণ্য, সীমা ৫০০	মোট 2020 টাকা, পরিমাণ 16; সবচেয়ে বড় অঙ্ক School Bag (950); বাদ পড়েছে ১টি
সীমা ৫০০	School Bag, Water Bottle
সীমা ৯৫০	শুধু School Bag (সীমার সমান অঙ্কও ধরা হয়)
ফাঁকা Cart `[]`	মোট 0 টাকা, পরিমাণ 0; কোনো Error নয়
পরিমাণ `0`, `101` বা `2.5`	পণ্যটি বাদ পড়ে
নাম ফাঁকা বা দাম ঋণাত্মক	পণ্যটি বাদ পড়ে

Companion Source Code-এ এই Project-এর ১৫টি Automated Test আছে। চালানোর Command `README.md`-তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`TypeError: Cannot read properties of undefined (reading 'name')`	Loop-এর শর্তে `<=`	`i < items.length`
মোট `NaN`	কোনো পণ্যের দাম বা পরিমাণ সংখ্যা নয় এবং যাচাই বাদ পড়েছে	`isValidItem`-এর `continue` Block আছে কি না দেখুন
মোট শুধু শেষ পণ্যের সমান	`total` Loop-এর ভেতরে তৈরি	Loop-এর আগে `let total = 0;`
নামের তালিকার শুরুতে কমা	ফাঁকা যাচাইয়ের Ternary নেই	`bigItems === '' ? ... : ...`
অগ্রহণযোগ্য পণ্যও মোটে ধরা হচ্ছে	`continue` নেই	`skipped++`-এর পরে `continue;`
Program আটকে যায়	`i++` বাদ পড়েছে	`for`-এর তৃতীয় অংশ দেখুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Cart-এর তথ্য Code-এর ভেতরে লেখা এবং সংখ্যা হিসেবেই দেওয়া; Text থেকে রূপান্তর এখানে করা হয়নি (অধ্যায় ২ ও ৩-এর কৌশল প্রয়োগ করে তা যোগ করা যায়)। প্রতিটি `item` যে একটি Object এবং তাতে `name` আছে, তা ধরে নেওয়া হয়েছে; তা না হলে Program Error দেবে। ছাড় ও Delivery-র হিসাব নেই। টাকার অঙ্ক পূর্ণসংখ্যায় ধরা হয়েছে।

৪.১৫ অনুশীলনী

প্রাথমিক স্তর

১. `for` Loop দিয়ে ১ থেকে ১০০ পর্যন্ত সংখ্যার যোগফল বের করুন।

২. `while` Loop দিয়ে ৫ থেকে ১ পর্যন্ত উল্টো গুনে শেষে `শুরু!` দেখান।

৩. `[3, 8, 11, 16, 19]` Array থেকে শুধু জোড় সংখ্যাগুলো দেখান।

৪. না চালিয়ে বলুন: `for (let i = 0; i < 5; i += 2)` কয়বার চলবে এবং `i`-এর মানগুলো কী হবে?

৫. নিচের Code-এ একটি Off-by-One Error আছে। ভুলটি ঠিক করুন, যাতে তিনটি নামই দেখায় এবং `undefined` না আসে।

names-bug.js · JavaScript — চলবে: Browser Console ও Node.js

```
const names = ['Sara', 'Rafi', 'Mina'];

for (let i = 1; i <= names.length; i++) {
  console.log(names[i]);
}
```

মধ্যম স্তর

৬. `[40, 75, 12, 98, 63]` Array-র সর্বোচ্চ মান এবং সেটির Index বের করুন।

৭. একটি Array-তে প্রথম `0` পাওয়ার আগ পর্যন্ত সংখ্যাগুলো যোগ করুন; `0` পেলে থেমে যান। `[5, 10, 0, 20]` দিয়ে পরীক্ষা করুন।

৮. Nested Loop দিয়ে নিচের মতো একটি ত্রিভুজ দেখান (চার সারি):

Output

```
*
**
***
****
```

৯. `countPassed` নামে একটি Function লিখুন, যা নম্বরের Array এবং পাস নম্বর নিয়ে কতজন উত্তীর্ণ তা ফেরত দেয়। ফাঁকা Array দিলে `0` ফেরত দেবে।

চ্যালেঞ্জ স্তর

১০. **সবচেয়ে সস্তা:** পণ্যের Array থেকে সবচেয়ে কম দামের পণ্যের নাম বের করার Function লিখুন। ফাঁকা Array দিলে `null` ফেরত দেবে।

১১. **বাদ পড়া চিহ্নিত:** পরিমাণ শূন্য বা তার কম হলে পণ্যের লাইনের জায়গায় `(বাদ)` দেখায়, এমন একটি Function লিখুন।

১২. **কত মাসে দ্বিগুণ:** একটি অঙ্ক প্রতি মাসে ৫ শতাংশ করে বাড়লে কত মাসে দ্বিগুণ হয়, তা `while` দিয়ে বের করুন। প্রতি মাসের শেষে অঙ্কটি `Math.round()` করুন, এবং নিরাপত্তা-সীমা হিসেবে সর্বোচ্চ ১০০০ মাস রাখুন।

৪.১৬ অনুশীলনীর সমাধান

সমাধান ১

sum-hundred.js · JavaScript — চলবে: Browser Console ও Node.js

```
let sum = 0;

for (let i = 1; i <= 100; i++) {
  sum += i;
}

console.log(sum);
```

Output

```
5050
```

সমাধান ২

countdown.js · JavaScript — চলবে: Browser Console ও Node.js

```
let count = 5;

while (count >= 1) {
  console.log(count);
  count--;
}

console.log('শুরু!');
```

Output

```
5
4
3
2
1
শুরু!
```

সমাধান ৩

even-only.js · JavaScript — চলবে: Browser Console ও Node.js

```
const numbers = [3, 8, 11, 16, 19];

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) {
    console.log(numbers[i]);
  }
}
```

Output

```
8
16
```

সমাধান ৪ — তিনবার। `i`-এর মান হয় `0`, `2` এবং `4`। এরপর `i` হয় `6`, তখন `6 < 5` মিথ্যা, তাই Loop থামে।

সমাধান ৫ — দুটি ভুল একসঙ্গে: শুরু `1` থেকে (প্রথম নাম বাদ) এবং শর্তে `<=` (শেষে `undefined`)। সঠিক সীমা `0` থেকে `< length`।

names-fixed.js · JavaScript — চলবে: Browser Console ও Node.js

```
const names = ['Sara', 'Rafi', 'Mina'];

for (let i = 0; i < names.length; i++) {
  console.log(names[i]);
}
```

Output

```
Sara
Rafi
Mina
```

সমাধান ৬

highest-index.js · JavaScript — চলবে: Browser Console ও Node.js

```
const values = [40, 75, 12, 98, 63];
let highest = values[0];
let highestIndex = 0;

for (let i = 1; i < values.length; i++) {
  if (values[i] > highest) {
    highest = values[i];
    highestIndex = i;
  }
}

console.log(`সর্বোচ্চ ${highest}, Index ${highestIndex}`);
```

Output

```
সর্বোচ্চ 98, Index 3
```

এখানে `i = 1` থেকে শুরু করা ইচ্ছাকৃত এবং সঠিক: প্রথম Element আগেই `highest`-এ নেওয়া হয়েছে, তাই সেটির সঙ্গে নিজেকে তুলনা করার দরকার নেই।

সমাধান ৭

until-zero.js · JavaScript — চলবে: Browser Console ও Node.js

```
const numbers = [5, 10, 0, 20];
let sum = 0;

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] === 0) {
    break;
  }

  sum += numbers[i];
}

console.log(sum);
```

Output

```
15
```

সমাধান ৮

triangle.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (let row = 1; row <= 4; row++) {
  let line = '';

  for (let star = 1; star <= row; star++) {
    line += '*';
  }

  console.log(line);
}
```

Output

```
*
**
***
****
```

ভেতরের Loop-এর সীমা বাইরের Counter-এর ওপর নির্ভর করে (`star <= row`), তাই প্রতিটি সারিতে একটি করে তারা বাড়ে।

সমাধান ৯

count-passed.js · JavaScript — চলবে: Browser Console ও Node.js

```
function countPassed(marks, passMark) {
  let passed = 0;

  for (let i = 0; i < marks.length; i++) {
    if (marks[i] >= passMark) {
      passed++;
    }
  }

  return passed;
}

console.log(countPassed([72, 28, 45, 33], 33));
console.log(countPassed([], 33));
```

Output

```
3
0
```

ফাঁকা Array-র `length` শূন্য, তাই Loop একবারও চলে না এবং শুরুর মান `0`-ই ফেরত যায়। আলাদা কোনো যাচাই লাগেনি।

সমাধান ১০

cheapest.js · JavaScript — চলবে: Browser Console ও Node.js

```
function cheapestName(items) {
  if (items.length === 0) {
    return null;
  }

  let cheapest = items[0];

  for (let i = 1; i < items.length; i++) {
    if (items[i].price < cheapest.price) {
      cheapest = items[i];
    }
  }

  return cheapest.name;
}

const items = [
  { name: 'Notebook', price: 120 },
  { name: 'Pen', price: 15 },
  { name: 'Bag', price: 950 },
];

console.log(cheapestName(items));
console.log(cheapestName([]));
```

Output

```
Pen
null
```

সমাধান ১১

mark-skipped.js · JavaScript — চলবে: Browser Console ও Node.js

```
function printLines(items) {
  for (let i = 0; i < items.length; i++) {
    const item = items[i];

    if (item.quantity <= 0) {
      console.log(`${i + 1}. ${item.name}: (বাদ)`);
      continue;
    }

    console.log(`${i + 1}. ${item.name}: ${item.price * item.quantity}`);
  }
}

printLines([
  { name: 'Pen', price: 15, quantity: 10 },
  { name: 'Calculator', price: 650, quantity: 0 },
]);
```

Output

```
1. Pen: 150
2. Calculator: (বাদ)
```

সমাধান ১২

double.js · JavaScript — চলবে: Browser Console ও Node.js

```
const MAX_MONTHS = 1000;
const start = 10000;
let amount = start;
let months = 0;

while (amount < start * 2 && months < MAX_MONTHS) {
  amount = Math.round(amount * 105 / 100);
  months++;
}

console.log(`${months} মাসে ${amount}`);
```

Output

```
15 মাসে 20791
```

নিরাপত্তা-সীমাটি এখানে শর্তের অংশ: অঙ্ক দ্বিগুণ হয়নি **এবং** সর্বোচ্চ মাস পেরোয়নি। শতাংশের হিসাবে আগে গুণ, পরে ভাগ করা হয়েছে।

৪.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `for` Loop-এর Update অংশ (যেমন `i++`) কখন চলে?

ক) Loop শুরুর আগে একবার

খ) প্রতিটি Iteration-এর পরে

গ) প্রতিটি Iteration-এর আগে

ঘ) শুধু শর্ত মিথ্যা হলে

প্রশ্ন ২. পাঁচটি Element-এর Array-তে শেষ Element-এর Index কত?

ক) `5`

খ) `6`

গ) `4`

ঘ) `0`

প্রশ্ন ৩. `do...while` Loop-এর ভেতরের Code সর্বনিম্ন কতবার চলে?

ক) একবার

খ) শূন্যবার

গ) দুইবার

ঘ) শর্তের ওপর নির্ভর করে

প্রশ্ন ৪. `break` কী করে?

ক) বর্তমান Iteration বাদ দিয়ে পরেরটিতে যায়

খ) Program বন্ধ করে দেয়

গ) Loop আবার শুরু থেকে চালায়

ঘ) Loop সম্পূর্ণ থামিয়ে দেয়

প্রশ্ন ৫. `continue` কী করে?

ক) Loop সম্পূর্ণ থামিয়ে দেয়

খ) বর্তমান Iteration-এর বাকি অংশ বাদ দিয়ে পরেরটিতে যায়

গ) Function থেকে মান ফেরত দেয়

ঘ) Counter শূন্য করে দেয়

প্রশ্ন ৬. `for (let i = 0; i <= arr.length; i++)` লিখলে কী সমস্যা হয়?

ক) প্রথম Element বাদ পড়ে

খ) Loop একবারও চলে না

গ) শেষে একটি অতিরিক্ত Iteration হয়, যেখানে `arr[i]` হলো `undefined`

ঘ) কোনো সমস্যা হয় না

প্রশ্ন ৭. কতবার চলতে হবে তা আগে থেকে জানা না থাকলে কোন Loop সবচেয়ে উপযুক্ত?

ক) `while`

খ) `for`

গ) `switch`

ঘ) Nested `for`

প্রশ্ন ৮. বাইরের Loop ৪ বার এবং ভেতরের Loop ৫ বার চললে ভেতরের Code মোট কতবার চলে?

ক) ৯ বার

খ) ৫ বার

গ) ৪ বার

ঘ) ২০ বার

প্রশ্ন ৯. নিচের কোনটি Infinite Loop তৈরি করে?

ক) `for (let i = 0; i < 5; i++) { }`

খ) `let i = 0; while (i < 5) { console.log(i); }`

গ) `let i = 5; while (i > 0) { i--; }`

ঘ) `for (let i = 5; i > 0; i--) { }`

প্রশ্ন ১০. সব সংখ্যা ঋণাত্মক হতে পারে, এমন একটি Array-র সর্বোচ্চ মান খুঁজতে শুরুর মান কী হওয়া উচিত?

ক) `0`

খ) `null`

গ) Array-র প্রথম Element

ঘ) `Infinity`

৪.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–খ · ৬–গ · ৭–ক · ৮–ঘ · ৯–খ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	ক্রমটি হলো: শর্ত, ভেতরের Code, Update, আবার শর্ত।
২	গ	Index শূন্য থেকে শুরু, তাই শেষ Index হলো `length - 1`।
৩	ক	`do...while` আগে চলে, পরে শর্ত দেখে।
৪	ঘ	`break`-এর পর Loop-এর পরের লাইন থেকে Program চলে।
৫	খ	Loop চলতে থাকে; শুধু এই Iteration-এর বাকি অংশ বাদ যায়।
৬	গ	`i` যখন `length`-এর সমান, তখন সেই Index-এ কিছু নেই।
৭	ক	`while` শুধু একটি শর্ত দেখে এবং সেটি সত্য থাকা পর্যন্ত চলে।
৮	ঘ	বাইরের প্রতিটি Iteration-এ ভেতরের Loop পুরোটা চলে: ৪ × ৫।
৯	খ	`i` কখনো বদলায় না, তাই `i < 5` সব সময় সত্য।
১০	গ	`0` দিয়ে শুরু করলে সব সংখ্যা ঋণাত্মক হলে ভুল করে `0`-ই সর্বোচ্চ থেকে যায়।

৪.১৯ অধ্যায়ের সারসংক্ষেপ

- Loop একটি কাজ বারবার চালায়; প্রতিবার চলা একটি Iteration।
- `for`-এ তিনটি অংশ: Initialization, Condition, Update। সংখ্যা জানা থাকলে বা Array ধরে চললে এটি ব্যবহার করুন।
- Array ক্রমানুসারে মান রাখে; Index শূন্য থেকে শুরু, শেষ Index `length - 1`।
- Array-র Loop: `let i = 0; i < array.length; i++`।
- Off-by-One Error আসে শুরুর মান বা শর্তের Operator থেকে; প্রথম ও শেষ Iteration যাচাই করুন।
- `while` শর্ত সত্য থাকা পর্যন্ত চলে; `do...while` অন্তত একবার চলে।
- `break` Loop থামায়; `continue` বর্তমান Iteration এড়িয়ে যায়।
- Nested Loop-এ কাজ গুণ হয়ে বাড়ে।
- Infinite Loop-এর কারণ: Update নেই, ভুল দিকে, অথবা শর্ত কখনো মিথ্যা হয় না। `Ctrl + C` দিয়ে থামান; নিরাপত্তা-সীমা দিয়ে প্রতিরোধ করুন।
- যোগফল, গণনা, সর্বোচ্চ, সর্বনিম্ন এবং খোঁজা—Loop-এর পাঁচটি মৌলিক ধরন, প্রতিটিতে একটি Accumulator থাকে।

পরের অধ্যায়ে Function এবং Scope: Code-কে পুনর্ব্যবহারযোগ্য অংশে ভাগ করা, এবং কোন Variable কোথা থেকে দেখা যায় তার নিয়ম।

৪.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Loop	একই Code বারবার চালানোর ব্যবস্থা।
Iteration	Loop-এর ভেতরের Code-এর একবার চলা।
Counter	Loop কতবার চলেছে তা ধরে রাখার Variable।
Array	ক্রমানুসারে একাধিক মান রাখার মান।
Element / Index	Array-র একটি মান / তার অবস্থানের নম্বর (শূন্য থেকে শুরু)।
Off-by-One Error	Loop একবার বেশি বা একবার কম চলার ভুল।
Nested Loop	একটি Loop-এর ভেতরে আরেকটি Loop।
Infinite Loop	যে Loop কখনো থামে না।
Accumulator	Loop জুড়ে ফল জমা রাখার Variable।
Trace Table	প্রতিটি Iteration-এ Variable-এর মান লিখে রাখার টেবিল।

৪.২১ আরও পড়ুন

- **MDN — Loops and iteration:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration` — সব ধরনের Loop-এর বিবরণ।
- **MDN — for:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for` — `for` Statement-এর পূর্ণ নিয়ম।
- **MDN — while:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/while` — `while` Statement।
- **MDN — break:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/break` — `break`-এর ব্যবহার।